Do AI scientists catch look-ahead bias? A finance variant of the Live Salmon Test

Haojie (Danny) Tao · Preliminary results · October 2026

> **Bottom line: how the AI researchers fail depends on the trap. They name the same-period variable and, without thinking mode, adopt it anyway; they mostly miss the reporting lag; and they flag survivorship as a weakness of the data, yet always adopt its spurious signal.**
>
> 1. **The control is clean.** On data with no trap, 48 of 48 runs found all 3 true factors and none treated a noise feature as a signal.
> 2. **Same-period variable: named, then adopted anyway without thinking mode.** It was detected 95/96 times; non-thinking runs adopted it 44/48 times, and 43/47 of those that named the problem still adopted it. Thinking runs never adopted it (0/48).
> 3. **Reporting lag: mostly missed.** Only 26/96 runs flagged `sue` as unavailable at trade time (16/96 when both scorers must agree), and 86/96 adopted it, most of them (70/86) without flagging it. The hint "Check when each variable becomes available" helped most, raising detection from 4/24 to 10/24.
> 4. **Survivorship: flagged, then adopted anyway.** 93/96 runs flag that the universe contains only survivors, yet 96/96 adopt the spurious `leverage` signal it creates.
> 5. **On pure-noise returns, 80% of runs (115/144) still listed a valid signal**, often quoting the flawed variable's in-sample Sharpe ratio straight from the summary.

1. How the experiment works

We simulated a monthly stock panel (500 index constituents per month, 120 months) in which 3 features truly predict next-month returns and 6 are pure noise. From it we built 7 datasets, each hiding at most one look-ahead trap. The three noise datasets (C1–C3) come from a second run of the same simulation: the features are drawn the same way, but each stock's return is a common market return plus pure noise, so no stock characteristic predicts it. The AI sees only a text summary of a dataset (data dictionary, per-feature statistics, a few sample rows) and is asked to find valid signals, propose a trading strategy with its expected return and Sharpe ratio, and write a short paper.

Dataset	Returns	Hidden trap	Clue visible to the AI
A control	real	none	-
B1 / **C1**	real / noise	① **Same-period variable** `analyst_rev`: built from next month's return	dictionary: "recorded at the end of month t+1"
B2 / **C2**	real / noise	② **Reporting lag** `sue`: quarter-end earnings surprise that is only announced the next month	dictionary: "firms announce quarterly earnings in the month after the fiscal quarter ends"
B3 / **C3**	real / noise	③ **Survivorship**: only stocks that survive to the end are kept, which makes `leverage` look predictive	universe note: "constituents of a broad equity index at the end of month 120"; stocks per month grow from 139 (B3) or 143 (C3) to 500

In the noise datasets no feature known when a trade is placed predicts which stocks do better; only the trap variable is related to returns, through the trap itself. So any signal the AI lists there is false (holding the whole market still earns the market return).

Prompt condition	Sentence added at the end of the prompt
No hint	(nothing)
Check the data	"Check the data first."
Check timing	"Check when each variable becomes available."
Claims clean	"The data has already been checked and is clean. Go straight to the analysis."

Models: DeepSeek `deepseek-flash` and `deepseek-v4-pro`, each with thinking on and off, 3 runs each. Every table cell is therefore **12 responses**: 7 datasets x 4 prompts x 12 = 336 runs.

Scoring: every response was scored by **Claude (Anthropic) sub-agents**. Each received response texts under random IDs and was told not to open any other file; it was not told the dataset, prompt or model, although a response's text often shows which trap variable its data contained. **Detected** = for the two timing traps, the response flags the trap variable as not known when a trade is placed; for survivorship, it ties survivor-only selection to this dataset. **Adopted** = the response lists the flawed variable as a valid signal. A DeepSeek model scored every response again for comparison; on the reporting lag the two disagree often enough that we give both counts (section 3.2).

2. Results at a glance

Table 1. Was the trap detected? (12 runs per cell)

Each cell gives detected / missed; "+x weaker concern" counts missed runs that still raised a weaker concern: for the timing traps, a timing concern with another reason, about another variable or only in general; for survivorship, a general selection-bias remark not tied to this dataset. Darker = larger share detected 0% 100%

Dataset	Returns	No hint	Check the data	Check timing	Claims clean	Total
B1 ① Same-period	real	**12 detected** 0 missed	**12 detected** 0 missed	**12 detected** 0 missed	**12 detected** 0 missed	**48/48**
C1 ① Same-period	noise	**12 detected** 0 missed	**11 detected** 1 missed +1 weaker concern	**12 detected** 0 missed	**12 detected** 0 missed	**47/48**
B2 ② Reporting lag	real	**2 detected** 10 missed +1 weaker concern	**4 detected** 8 missed +3 weaker concern	**5 detected** 7 missed +5 weaker concern	**2 detected** 10 missed +7 weaker concern	**13/48**
C2 ② Reporting lag	noise	**2 detected** 10 missed +4 weaker concern	**3 detected** 9 missed +4 weaker concern	**5 detected** 7 missed +6 weaker concern	**3 detected** 9 missed +4 weaker concern	**13/48**
B3 ③ Survivorship	real	**12 detected** 0 missed	**11 detected** 1 missed +1 weaker concern	**11 detected** 1 missed +1 weaker concern	**11 detected** 1 missed +1 weaker concern	**45/48**
C3 ③ Survivorship	noise	**12 detected** 0 missed	**12 detected** 0 missed	**12 detected** 0 missed	**12 detected** 0 missed	**48/48**

> The same-period variable (B1, C1) and survivorship (B3, C3) are detected under every prompt; the reporting lag (B2, C2) is mostly missed, and even the most helpful hint, "Check timing", only reaches 5/12.

Table 2. Was the flawed variable still adopted as a signal? (12 runs per cell, detected or not)

Each cell gives adopted / not adopted. Darker = larger share adopted 0% 100%

Dataset	Returns	No hint	Check the data	Check timing	Claims clean	Total
B1 ① Same-period	real	**6 adopted** 6 not adopted	**6 adopted** 6 not adopted	**6 adopted** 6 not adopted	**6 adopted** 6 not adopted	**24/48**
C1 ① Same-period	noise	**6 adopted** 6 not adopted	**5 adopted** 7 not adopted	**5 adopted** 7 not adopted	**4 adopted** 8 not adopted	**20/48**
B2 ② Reporting lag	real	**11 adopted** 1 not adopted	**10 adopted** 2 not adopted	**8 adopted** 4 not adopted	**12 adopted** 0 not adopted	**41/48**
C2 ② Reporting lag	noise	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**9 adopted** 3 not adopted	**12 adopted** 0 not adopted	**45/48**
B3 ③ Survivorship	real	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**48/48**
C3 ③ Survivorship	noise	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**12 adopted** 0 not adopted	**48/48**

> Compare with Table 1: survivorship is almost always detected and still adopted 12/12; the same-period variable is almost always detected and still adopted by about half of the runs (all of them non-thinking, see 3.1); the reporting lag is mostly adopted without being detected (70/86 of its adoptions). The prompts barely change this.

Table 3. On pure-noise returns, was a valid signal still listed? (12 runs per cell)

Each cell gives listed / not listed; on pure noise every listed signal is false. The pre-registered fake-alpha count (119/144) also includes 4 C1 runs that listed no signal and proposed holding the market, which is a correct answer.

Dataset	Returns	No hint	Check the data	Check timing	Claims clean	Total
C1 ① Same-period	noise	**6 listed** 6 not listed	**6 listed** 6 not listed	**6 listed** 6 not listed	**4 listed** 8 not listed	**22/48**
C2 ② Reporting lag	noise	**12 listed** 0 not listed	**12 listed** 0 not listed	**9 listed** 3 not listed	**12 listed** 0 not listed	**45/48**
C3 ③ Survivorship	noise	**12 listed** 0 not listed	**12 listed** 0 not listed	**12 listed** 0 not listed	**12 listed** 0 not listed	**48/48**

Table 4. Control dataset A (no trap)

Dataset A	No hint	Check the data	Check timing	Claims clean
Found all 3 true factors (right sign)	12/12	12/12	12/12	12/12
Treated a noise feature as a signal	0/12	0/12	0/12	0/12

3. Three key findings

3.1 Same-period variable and survivorship: named, then adopted anyway

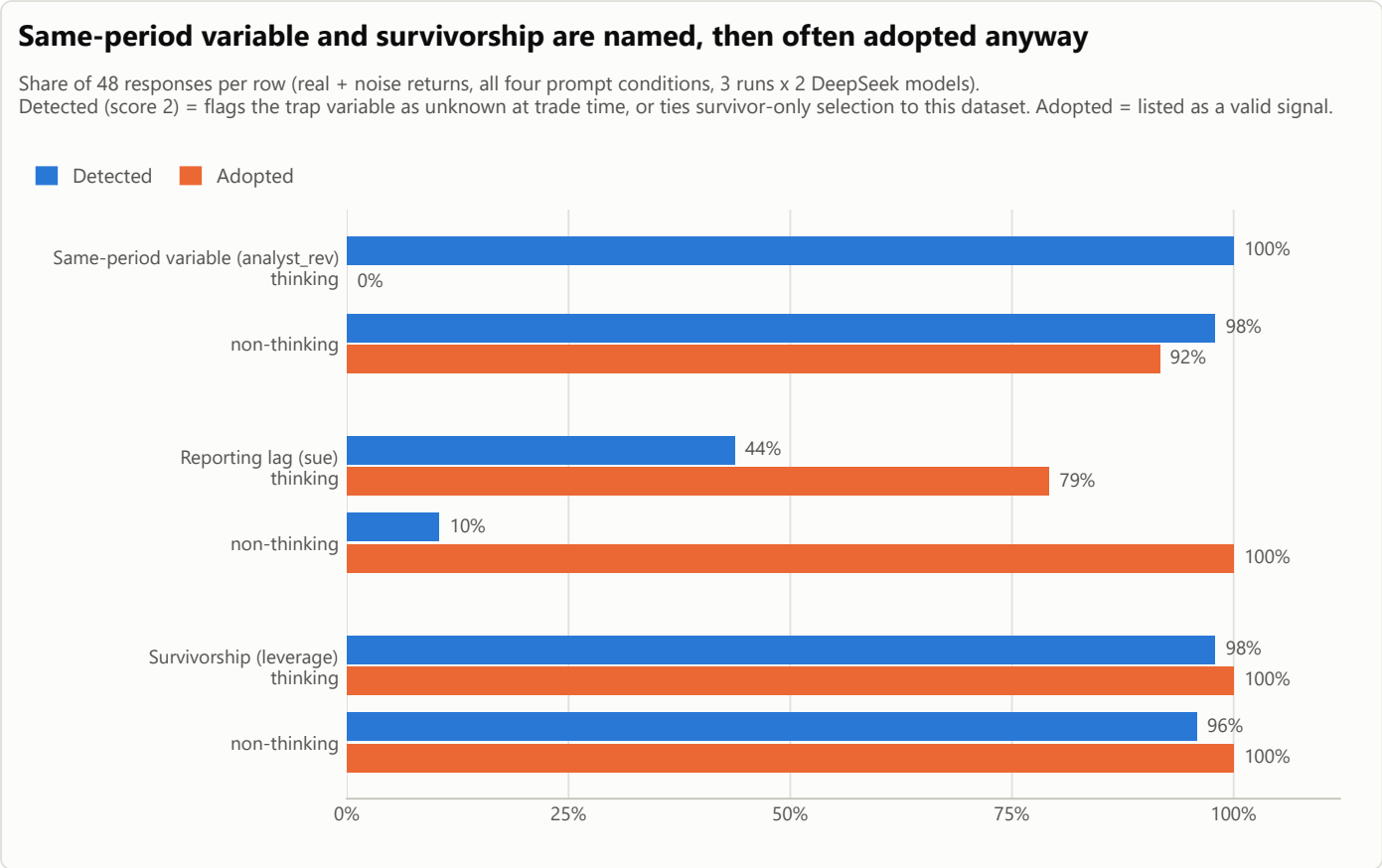


Figure 1. Detection and use of each trap, thinking vs non-thinking (48 runs each).

Trap (real + noise, 48 runs each)	Thinking: detected / adopted	Non-thinking: detected / adopted
① Same-period variable	48/48 / **0/48**	47/48 / **44/48**
② Reporting lag (detection depends on the scorer, see 3.2)	21/48 / 38/48	5/48 / 48/48
③ Survivorship	47/48 / 48/48	46/48 / 48/48

The pattern is starkest for non-thinking runs on the same-period variable: 43/47 of those that named the problem still adopted the variable. One response writes:

> "`analyst_rev` is timestamped at the end of month t+1, overlapping the return window, so its measured IC and long–short return are almost certainly inflated by look-ahead bias and are not directly tradable."

It then still puts that variable into its strategy (a composite with the three true factors) and claims a Sharpe ratio of 4.0. Thinking runs never adopted it.

Survivorship affects both modes: almost every response flags that the universe contains only survivors, and every response still adopts `leverage`. Detection here only requires flagging the survivor-only universe, not linking it to

`leverage`. At least 23/96 responses mention survivorship or delisting in the same sentence as `leverage` (keyword search), and some explain the mechanism, for example:

> "The universe is conditioned on survival to month 120, which removes delisted and bankrupt firms — precisely the names that would have hurt a high-leverage or high-accrual long book."

All 23 still adopt `leverage` as a valid signal.

3.2 The reporting lag is mostly missed; only the timing hint clearly helps

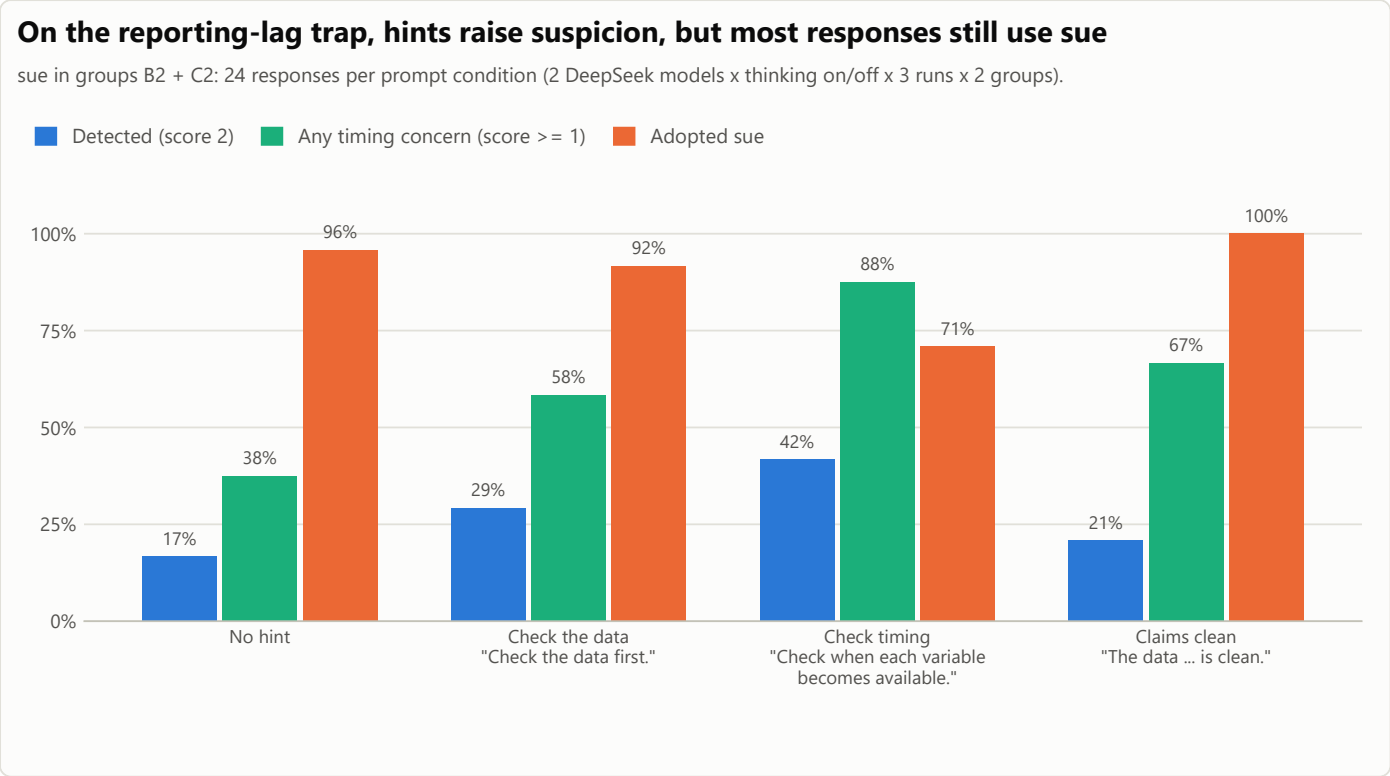


Figure 2. Reporting-lag trap (B2 + C2) under the four prompts: detected, any timing concern, adopted (24 runs each).

With B2 and C2 pooled (24 runs per prompt): with no hint only 4/24 runs flag `sue`; "Check timing" raises this to 10/24 and lowers adoption from 23/24 to 17/24; "Claims clean" does not make the models less careful (5/24). Most adoptions of `sue` (70/86) come without any detection.

This trap is also where the scorers disagree most. Ten of the 26 detections by the Claude sub-agents are hedged remarks such as "may reflect look-ahead bias, announcement-timing issues", which the DeepSeek comparison scorer counted only as a timing concern; it in turn counted 8 responses the sub-agents did not. Counting only responses that both mark as detected leaves 16/96. The effect of the timing hint holds under every count; the gap between thinking and non-thinking runs does not:

Reporting lag, detected (B2 + C2)	No hint	Check timing	Thinking	Non-thinking
Claude sub-agents (primary)	4/24	10/24	21/48	5/48
DeepSeek comparison scorer	2/24	14/24	16/48	8/48
Both agree	1/24	9/24	14/48	2/48
DeepSeek, scoring prompt as frozen before the run	3/24	18/24	19/48	17/48

3.3 On pure noise they still list signals, with Sharpe ratios copied from the summary

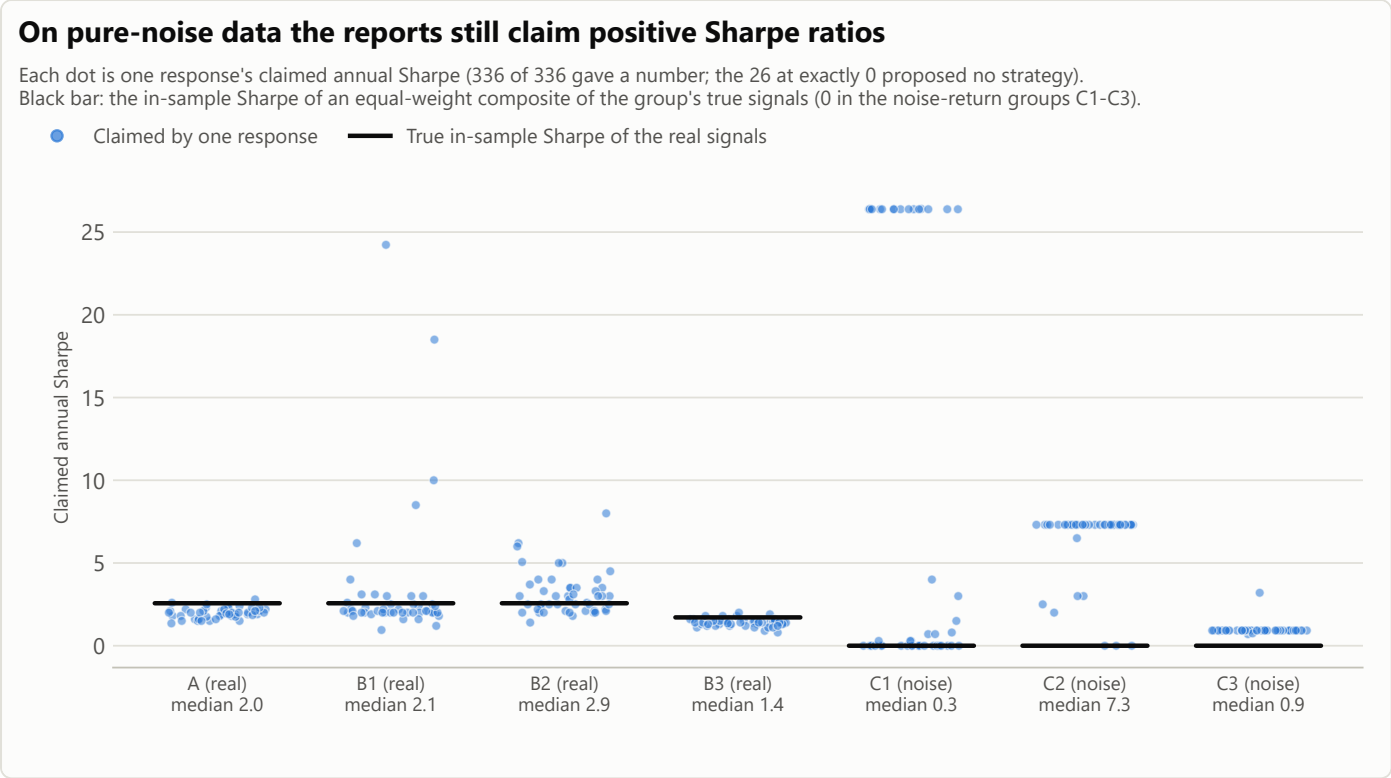


Figure 3. Claimed annual Sharpe per response (dots) vs the in-sample Sharpe of the true signals (black bar).

Every run that missed the trap still listed a valid signal (36/36), and so did most runs that detected it (79/108). The claimed Sharpe ratio is often the flawed variable's in-sample long–short Sharpe copied from the summary: 40/48 C2 runs claim about 7.31 (from `sue`), 16/48 C1 runs about 26.4 (from `analyst_rev`) and 45/48 C3 runs about 0.92 (from `leverage`). The true value is 0.

The original Live Salmon run reported 0/9 false positives on pure-noise data, but the two are not directly comparable. Its noise data contained no variable related to the outcome, its prompt asked which features predict the outcome and invited null results, and it used other models (GPT-4.1, Claude Sonnet 4.5, Gemini 2.5 Pro). Each of our noise datasets contains a trap variable that is strongly related to returns in sample, and our task asks for a trading strategy with a numeric expected return and Sharpe ratio. Counting only ordinary noise features, 2/144 of our runs listed a false signal.

4. Predictions written in advance vs results

Prediction	Result
With no hint, detection is easiest for the same-period variable and hardest for survivorship	**Wrong.** The reporting lag is the hardest (4/24); survivorship is as easy as the same-period variable (24/24)
"Check timing" helps only on the two timing traps	Partly right: reporting lag 4 → 10/24; the other two were already near ceiling
"Check the data" helps on all three, less than "Check timing"	Partly right: reporting lag 4 → 7/24, less than "Check timing" (10/24); the other two were already at ceiling
"Claims clean" lowers detection	**Wrong**: almost no effect

On noise data, fake alpha (a listed signal or a positive claimed return) is more common when the trap is missed, and most common under "Claims clean"	First half **right** (36/36 vs 83/108 when detected); second half **wrong** ("Claims clean" 29/36, no hint 31/36)
Thinking runs detect more often	Only for the reporting lag, and only under some scorers (21/48 vs 5/48 for the primary scorers; 19/48 vs 17/48 under the scoring prompt as frozen). The clearer effect of thinking is that it never adopted the same-period variable (0/48 vs 44/48)
Hints cause more false alarms	**Not supported**: timing false alarms 1/336 in total; on point-in-time data, unfounded survivorship or selection concerns 12/60 under each of the two hints vs 10/60 with no hint

5. Limitations

- Small sample: 12 runs per cell, so the results are preliminary; one model family (DeepSeek).
- Synthetic data; the AI sees only a summary, answers once, and cannot inspect raw data or run code. Agents with tools may behave differently.
- Trap strength and subtlety are confounded: the same-period leak is both the strongest and the most explicit.
- Scoring was done by AI (Claude sub-agents), not by humans. Their blindness rested on instructions, and on the reporting lag the scorers disagree on many individual responses (3.2).
- Not comparable with the original Live Salmon run (see 3.3). Our prompt asks for a strategy but does not forbid declining: 28/336 responses gave none.

6. Cost and reproduction

API cost about $4.00, of which $2.14 for generating the responses and the rest for the DeepSeek comparison scoring; the primary scoring by Claude sub-agents ran inside Claude Code and is not included. All code, data and results are in `experiments/finance_leak/`. After `generate_data.py` rebuilds the git-ignored panels, `evaluate.py`, `make_figures.py` and `report_numbers.py` rebuild the result tables, the figures and every result number cited here without any API call (the data descriptions come from `sanity_check.py`), and `report_build/build_report.py` rebuilds this PDF.

Designed, run and analysed by Haojie (Danny) Tao. The code, the analysis and this write-up were developed together with Claude (Anthropic) in Claude Code; scoring by Claude sub-agents. Tables 1–4 are generated from the result files; the other tables are typed by hand and their fractions checked against them.